TECHNICAL REFERENCE

Storage Rack

Data Tables, Modules, Services & API Surface

Inventory & storage-location management application

Client-side single-page app set (vanilla HTML / CSS / JavaScript)

Generated 2026-10-13

1. Architecture Overview

Storage Rack is a fully client-side application — there is no backend server, database, or network API. Every page loads `js/common.js` (the shared data + UI layer) plus a page-specific script. All data is persisted in the browser's `localStorage` as JSON, keyed by a set of versioned keys (e.g. `srStorageV4`). Pages communicate only through that shared storage and a small set of global callback hooks (`window.onRackDataChanged`, `window.onItemDataChanged`) that each page's render function subscribes to.

> Because there is no server, the sections below use "Tables" to mean the **localStorage data collections** that stand in for database tables, and "APIs" to mean the **internal JavaScript function surface** that each page module calls into `common.js` — there are no REST/HTTP endpoints in this project.

2. Data Tables (localStorage collections)

Six JSON collections make up the entire data model. Each is loaded once per page load, held in a module-level variable, and re-saved on every mutation.

2.1 Storage Units — `racks` (key: srStorageV4)

FIELD	TYPE	DESCRIPTION
id	string	Unique ID, e.g. `ST-1008`
name	string	Display name, e.g. "Chem Storage"
storeRoomId	string	FK → Locations.id
type	string	Storage type (rack, cupboard, bureau, wardrobe, shelf, cabinet, box, custom)
tags	string[]	Free-form category tags
images	string[]	Base64 data-URI pictures
nodes	array	Legacy sub-structure tree (deprecated — storages are flat; items attach directly)
createdAt	number	Epoch ms

2.2 Items — `globalItems` (key: srGlobalItemsV1)

FIELD	TYPE	DESCRIPTION
id	string	Unique ID, e.g. `IT-1013`
name	string	Item name
tags	string[]	Category tags
quantity	number	Units on hand
remarks	string	Free-text notes
storageId	string | null	FK → Storage Units.id (null = unassigned)
images	string[]	Base64 data-URI pictures
status	string	`in_store` | `in_use` | `removed` (archived)
createdAt	number	Epoch ms

2.3 Locations — `rooms` (key: srRoomsV1)

FIELD	TYPE	DESCRIPTION
id	string	Unique ID, e.g. `SR-1002`
name	string	Location name
location	string	Physical area / warehouse
createdAt	number	Epoch ms

2.4 Users — `users` (key: srUsersV1)

FIELD	TYPE	DESCRIPTION
id	string	Unique ID, e.g. `US-1002`
name	string	Full name
username	string	Login handle
email	string	Contact email
roleId	string	FK → Roles.id
status	string	active | inactive
createdAt	number	Epoch ms

2.5 Roles & Permissions — `roles` (key: srRolesV1)

FIELD	TYPE	DESCRIPTION
id	string	Unique ID, e.g. `RL-1002`
name	string	Role name
description	string	Free-text summary
permissions	object	Per-module CRUD matrix — keys: storage, items, rooms, users, roles; each holds {create, read, update, delete: boolean}
createdAt	number	Epoch ms

2.6 IN/OUT Log — `itemLog` (key: srItemLogV1)

FIELD	TYPE	DESCRIPTION
id	string	Unique log entry ID
timestamp	number	Epoch ms
type	string	in | out | in_store | in_use | removed
itemName	string	Item name at time of entry
quantity	number	Units moved
rackId / rackName	string	Storage unit involved (denormalized)
path	string | null	Legacy sub-structure path (from the deprecated nested-compartment tree); every call site in the current codebase passes `null`, so this field is effectively dead today but still present on every row
user	string	Acting user's *username* (from `sessionStorage`) — not a foreign key to Users.id

3. Modules (Pages)

Each HTML page pairs with one page-specific script, on top of the shared `common.js`, `toast.js` and `theme.js` loaded everywhere.

Login

`index.html` · `js/login.js`

Demo credential sign-in, "forgot password" and "reset demo data" flows.

Overview (Dashboard)

`dashboard.html` · `js/overview.js`

Summary stat cards, empty-storage alerts, recently added storages. Empty-state padding was tightened app-wide (60px → 32px) so panels with no content (e.g. "Empty Storage Units" when there are none) no longer reserve an oversized, unattractive blank box.

Items

`items.html` · `js/items.js`

Full items table: filter by Type and Status, sortable columns (including a dedicated Item ID column), search across item name, item ID, storage name, storage ID, tags and location, add/edit modal, archive & restore, CSV/XLSX import, CSV/PDF export. Tags render as colored chip widgets (matching Status's badge styling) rather than plain text. The item's picture has its own dedicated first column instead of being embedded inline with the name; items with no picture show a colored letter-avatar (the item name's first letter) in that column instead.

Storages

`storage.html` · `js/storage.js`

Storage unit card grid: filter/sort/search, pagination, add/edit/delete. Every card is a fixed, uniform height (260px) regardless of content — both the item-chip preview (`buildItemsPreviewHtml` , capped at 2 + "+N more") and the tag list (`rackCardTagsHtml` , capped at 2 + "+N") are kept tight so sparse cards (1 tag, 1 item) don't leave excess dead space while cards with lots of items/tags still truncate safely inside the same fixed box. The "N items" footer is pinned to the bottom of every card via a corrected flex chain (`.rack-card-body` now properly stretches so `.rack-meta` 's `margin-top: auto` — previously a no-op — actually pushes it down).

Sort: Name (A-Z / Z-A) | Sort: Newest First | Sort: Oldest First | Sort: Location (A-Z) | Sort: Type (A-Z) | Sort: Items (most / fewest)

Storage Detail

`storage-detail.html` · `js/storage-detail.js`

Single storage's page: per-field click-to-edit (name, location, tags, pictures), always-on QR code with print, items panel with inline row editing plus its own search, status filter and sortable column headers (Item, Tags, Status, Qty). Panel's Tags column renders as colored chip widgets, matching Status's badge styling. Items Stored, Type and Tags are each shown as their own separate widget card, matching the Item Detail page's layout. The panel also has a dedicated first Image column — a real thumbnail when the item has a picture, otherwise a colored letter-avatar — matching the Items page.

Item Detail

`item-detail.html` · `js/item-detail.js`

Single item's page: per-field click-to-edit (name, storage, tags, quantity, remarks, pictures). Quantity, Status and Tags are each shown as their own separate widget card, now spanning the full row width evenly (no more capped-width cards leaving dead space); activity history with filters; archive/restore.

IN/OUT Log

`log.html · js/log.js`

Full movement history: filter/sort/search, CSV export.

Locations

`storerooms.html · js/storerooms.js`

Manage physical locations storages belong to; add/edit/delete.

Users

`users.html · js/users.js`

User directory with role assignment; add/edit/delete. Each row has a colored letter-avatar (first letter of the user's name) as its first column, for visual identity consistent with the Items table.

Roles & Permissions

`roles.html · js/roles.js`

CRUD permission matrix per module; add/edit/delete roles.

Shared infrastructure

FILE	PURPOSE
js/common.js	Data layer, icon system, shared UI builders, modals (confirm, add item, quick-add storage/location, lightbox)
js/toast.js	Toast notifications + cross-page flash messages
js/theme.js	Light/dark theme toggle, applied before paint
js/vendor/qrcode.js	QR code generation (kazuhikoarase qrcode-generator)
js/vendor/xlsx.js	Spreadsheet import/export (SheetJS)
js/vendor/jspdf.js + jspdf.autotable.js	PDF export for item/log tables

4. Services (Shared Business Logic)

Functions in `js/common.js` that encapsulate data rules and are reused across multiple pages, grouped by concern.

Data persistence (load / save per table)

loadRacks / saveRacks · loadGlobalItems / saveGlobalItems · loadRoles / saveRoles · loadLog / saveLog · loadRooms / saveRooms · loadUsers / saveUsers

Lookups & ID generation

findRack · findGlobalItem · findRoom / roomLabel · nextRoomId / nextUserId / nextRoleId / nextItemCode / nextNodeCode · findRole / roleLabel · findUser · rackDetailUrl / itemDetailUrl · rackTags / rackImages · runIdSearch

Item lifecycle rules

archiveGlobalItem · restoreGlobalItem · purgeGlobalItem · activeGlobalItemsForRack · globalItemsForRack · unitItemCount · logTransaction (in/out) · logStatusChange

Storage deletion & permissions

deleteRack · emptyPermissions · permissionSummary

Shared modals (dynamically built, reused across pages)

confirmDialog · openAddItemModal / closeAddItemModal · openQuickAddStorageModal · openQuickAddLocationModal · openEditModal / closeEditModal · openLightbox / buildLightbox · openMoveItemModal / closeMoveItemModal

Two searchable "item picker" dropdowns exist, both matching on item name, ID, tag, remarks or current storage location, both rendering their results inline below the field (not as an absolutely-positioned overlay, so they can't be clipped by a modal's own scroll container):

- **Add Storage** (`openQuickAddStorageModal`) — a dedicated "Add Existing Items" field: multi-select via removable chips; on submit, every picked item's `storageId` is reassigned to the newly created storage.
- **Add Item** (`openAddItemModal`) — the **Item Name** field itself doubles as the search box (only while creating — suppressed while editing an existing item): typing shows matching existing items below it; clicking, or pressing Enter/arrow-keys to navigate and Enter to confirm, opens a `confirmDialog` ("Move X into Y? It will be taken out of Z.") before reassigning that item's `storageId` to whatever Storage is currently selected in the form — nothing moves until the user confirms. Pressing Enter with the dropdown open always targets the highlighted (or first) match instead of submitting the form, so it can't create an accidental duplicate item. Leaving the dropdown alone and just filling in the form still creates a brand-new item exactly as before.

Reusable UI builders

buildRackCard / buildRackRow · buildItemsPreviewHtml · populateRackSelect / roomOptionsHtml · tagsHtml / categoryBadgeHtml / typeBadgeHtml · renderPagination / paginateArray · itemStatusBadgeHtml · imagesPreviewHtml / rackQrThumbHtml · animateNumber / escapeHtml / svgIcon · createTagInput / createImageInput · getCategoryColor / getItemColor / hashString

5. API Surface (Internal, Client-Side)

> This project has **no network API** — no REST, GraphQL, or server routes exist. The table below documents the de-facto "API" each page module relies on: the shared functions and global callback hooks exposed by `common.js`, which act as the contract between modules.

CONTRACT	DIRECTION	USED BY
`window.onRackDataChanged`	Hook set by the current page's render function; called by common.js after any rack mutation (add/edit/delete, quick-add)	storage.html, storage-detail.html
`window.onItemDataChanged`	Hook set by the current page's render function; called by common.js after any item mutation (add/edit/archive/restore/purge/move)	items.html, storage-detail.html, item-detail.html
`findRack(id) / findGlobalItem(id) / findRoom(id) / findUser(id) / findRole(id)`	Synchronous read API — every page queries in-memory collections through these instead of touching localStorage directly	All pages
`save*(data)` family	Synchronous write API — the only sanctioned way to persist a table back to localStorage	All pages
`open*Modal(...)` family	UI API — pages invoke these instead of building their own modal markup, keeping modal behavior consistent app-wide	items.html, storage.html, storage-detail.html, item-detail.html
`confirmDialog(message, onConfirm, options)`	UI API replacing native `confirm()` everywhere in the app	All pages with destructive actions
`showToast(message, variant)` / `setFlashMessage`	Notification API (toast.js) — immediate toast or a flash message persisted across a redirect	All pages

6. Backend Development Guide

The app is currently 100% client-side (Section 1) — all six collections in Section 2 live only in one browser's `localStorage`, so nothing is shared between users or devices, and the "login" in `js/login.js` is a single hardcoded demo credential (`admin` / `admin123`) with no real session, token, or server-side permission check anywhere. The Roles & Permissions CRUD matrix (Section 2.5) is fully modeled and editable in the UI today, but nothing currently reads or enforces it — it's display-only until a backend exists to check it on every request. This section proposes a Django backend that preserves the existing data shapes (Section 2) and function contracts (Section 4–5) as closely as possible, to minimize required frontend rewrite.

6.1 Django Models

An **exact** field-for-field mirror of the six tables in Section 2 — every field name, type, and nullability matches the frontend's actual JSON exactly, with no added/dropped/upgraded fields. `id` stays the same string the frontend already generates (e.g. `ST-1008` , `IT-1013`) as the primary key rather than switching to auto-incrementing integers, so every existing `rackDetailUrl(id)` / `itemDetailUrl(id)` link, QR code payload, and CSV export keeps working untouched. `createdAt` / `timestamp` are `BigIntegerField` storing the exact same epoch-millisecond number `Date.now()` produces client-side — not Django's `DateTimeField` , which would change the wire format. `images` stays a plain `JSONField(list)` of base64 strings, identical to how the frontend already stores it — no file-backed model, no upload endpoint required to get started.

```python
# models.py
import time
from django.db import models


def epoch_ms():
    """Matches the frontend's Date.now() exactly — epoch milliseconds."""
    return int(time.time() * 1000)


class Location(models.Model):
    id = models.CharField(max_length=20, primary_key=True)        # e.g. "SR-1002"
    name = models.CharField(max_length=255)
    location = models.CharField(max_length=255, blank=True, default="")
    created_at = models.BigIntegerField(default=epoch_ms)

    class Meta:
        db_table = "locations"

    def __str__(self):
        return self.name


PERMISSION_MODULES = ["storage", "items", "rooms", "users", "roles"]

class Role(models.Model):
    id = models.CharField(max_length=20, primary_key=True)        # e.g. "RL-1002"
    name = models.CharField(max_length=255)
    description = models.TextField(blank=True, default="")
    # Exact shape the frontend already edits: {"storage": {"create":T,"read":T,"update":T,"delete":T}, ...}
    permissions = models.JSONField(default=dict)
    created_at = models.BigIntegerField(default=epoch_ms)

    class Meta:
        db_table = "roles"


class User(models.Model):
    id = models.CharField(max_length=20, primary_key=True)        # e.g. "US-1002"
    name = models.CharField(max_length=255)
    username = models.CharField(max_length=150, unique=True)
    email = models.EmailField(blank=True, default="")
    role = models.ForeignKey(Role, on_delete=models.SET_NULL, null=True, blank=True, related_name="users")
    status = models.CharField(
        max_length=20,
        choices=[("active", "Active"), ("inactive", "Inactive")],
        default="active",
    )
    created_at = models.BigIntegerField(default=epoch_ms)

    class Meta:
        db_table = "users"


STORAGE_TYPES = [
    ("rack", "Rack"), ("cupboard", "Cupboard"), ("bureau", "Bureau"),
    ("wardrobe", "Wardrobe"), ("shelf", "Shelf"), ("cabinet", "Cabinet"),
    ("box", "Box"), ("custom", "Custom"),
]

class StorageUnit(models.Model):
    id = models.CharField(max_length=20, primary_key=True)        # e.g. "ST-1008"
    name = models.CharField(max_length=255)
    store_room = models.ForeignKey(Location, on_delete=models.PROTECT, related_name="storage_units")
    type = models.CharField(max_length=20, choices=STORAGE_TYPES)
    tags = models.JSONField(default=list)                         # list[str], exact match
    images = models.JSONField(default=list)                       # list[str] of base64 data-URIs, exact match
    created_at = models.BigIntegerField(default=epoch_ms)

    class Meta:
        db_table = "storage_units"
```

```
    def __str__(self):
        return f"{self.id} {self.name}"


ITEM_STATUS = [("in_store", "In Store"), ("in_use", "In Use"), ("removed", "Removed")]

class Item(models.Model):
    id = models.CharField(max_length=20, primary_key=True)        # e.g. "IT-1013"
    name = models.CharField(max_length=255)
    tags = models.JSONField(default=list)
    quantity = models.PositiveIntegerField(default=1)
    remarks = models.TextField(blank=True, default="")
    storage = models.ForeignKey(
        StorageUnit, on_delete=models.SET_NULL, null=True, blank=True, related_name="items"
    )
    images = models.JSONField(default=list)                       # list[str] of base64 data-URIs, exact match
    status = models.CharField(max_length=20, choices=ITEM_STATUS, default="in_store")
    created_at = models.BigIntegerField(default=epoch_ms)

    class Meta:
        db_table = "items"

    def __str__(self):
        return f"{self.id} {self.name}"


LOG_TYPES = [
    ("in", "In"), ("out", "Out"),
    ("in_store", "In Store"), ("in_use", "In Use"), ("removed", "Removed"),
]

class ItemLogEntry(models.Model):
    id = models.CharField(max_length=30, primary_key=True)
    timestamp = models.BigIntegerField(default=epoch_ms)          # epoch ms, exact match
    type = models.CharField(max_length=20, choices=LOG_TYPES)
    item_name = models.CharField(max_length=255)                  # itemName — denormalized snapshot, exact match
    quantity = models.PositiveIntegerField()
    rack = models.ForeignKey(
        StorageUnit, on_delete=models.SET_NULL, null=True, blank=True, related_name="log_entries"
    )
    rack_name = models.CharField(max_length=255, blank=True, default="")   # rackName — denormalized snapshot
    path = models.CharField(max_length=500, null=True, blank=True)         # legacy field, always null today — kept for exact
parity
    user = models.CharField(max_length=150)                       # exact match: raw username string, not a FK

    class Meta:
        db_table = "item_log"
        ordering = ["-timestamp"]
        verbose_name_plural = "item log entries"
```

> **Where this model set is *not* a 1:1 JSON mirror, and why:** `store_room` / `storage` / `rack` / `role` are Django `ForeignKey`s rather than plain ID strings — that's the direct relational-database equivalent of the frontend's `storeRoomId` / `storageId` / `rackId` / `roleId` string fields (the serializers in Section 6.2 expose them back out as those exact same ID strings, so nothing changes from the frontend's point of view). The frontend's Users table (Section 2.4) has **no password field at all** — `js/login.js` hardcodes a single demo credential unrelated to the Users data — so a real login needs *adding* a password field (e.g. Django's own `AbstractUser`, or a bolted-on `password_hash = models.CharField(max_length=255)`); that's new, not something the current app already has and this doc silently dropped.

6.2 DRF Serializers

One serializer per model, with explicit field renames (`source=`) so the JSON over the wire uses the exact same camelCase field names the frontend already expects (`storeRoomId`, `storageId`, etc.) even though the Python/DB side stays snake_case.

```python
# serializers.py
from rest_framework import serializers
from .models import Location, Role, User, StorageUnit, Item, ItemLogEntry


class LocationSerializer(serializers.ModelSerializer):
    class Meta:
        model = Location
        fields = ["id", "name", "location", "created_at"]


class RoleSerializer(serializers.ModelSerializer):
    class Meta:
        model = Role
        fields = ["id", "name", "description", "permissions", "created_at"]


class UserSerializer(serializers.ModelSerializer):
    roleId = serializers.CharField(source="role_id", required=False, allow_null=True)

    class Meta:
        model = User
        fields = ["id", "name", "username", "email", "roleId", "status", "created_at"]


class StorageUnitSerializer(serializers.ModelSerializer):
    storeRoomId = serializers.CharField(source="store_room_id")
    # images is already a plain JSONField(list) on the model — ModelSerializer
    # passes it through as-is, exactly like tags, with no nested serializer needed.

    class Meta:
        model = StorageUnit
        fields = ["id", "name", "storeRoomId", "type", "tags", "images", "created_at"]


class ItemSerializer(serializers.ModelSerializer):
    storageId = serializers.CharField(source="storage_id", required=False, allow_null=True)

    class Meta:
        model = Item
        fields = ["id", "name", "tags", "quantity", "remarks", "storageId", "images", "status", "created_at"]


class ItemLogEntrySerializer(serializers.ModelSerializer):
    itemName = serializers.CharField(source="item_name")
    rackId = serializers.CharField(source="rack_id", allow_null=True)
    rackName = serializers.CharField(source="rack_name")

    class Meta:
        model = ItemLogEntry
        fields = ["id", "timestamp", "type", "itemName", "quantity", "rackId", "rackName", "path", "user"]
```

6.3 Permission Enforcement

The Roles & Permissions matrix (Section 2.5) is fully modeled and editable in the UI today, but nothing currently reads or enforces it — this class is what finally makes it do something, by checking the requesting user's role against the exact `{module: {create,read,update,delete}}` JSON shape already in `Role.permissions`.

```
# permissions.py
from rest_framework.permissions import BasePermission

ACTION_TO_PERM = {
    "list": "read", "retrieve": "read",
    "create": "create", "update": "update", "partial_update": "update",
    "destroy": "delete",
}

class HasModulePermission(BasePermission):
    """Checks request.user.role.permissions[view.module][action] — the exact
    JSON structure the Roles & Permissions page already edits (Section 2.5)."""

    def has_permission(self, request, view):
        if not request.user or not request.user.is_authenticated:
            return False
        module = getattr(view, "module", None)
        if not module:
            return True
        role = request.user.role
        if not role:
            return False
        perm_key = ACTION_TO_PERM.get(view.action, "read")
        return bool(role.permissions.get(module, {}).get(perm_key))
```

6.4 DRF Views

One `ModelViewSet` per resource. `StorageUnitViewSet` and `ItemViewSet` add the same search/filter query params the frontend's `applyRacksFilters` / `applyItemFilters` already compute client-side (Section 4), just moved server-side. `ItemViewSet.archive` / `.restore` / `.move` are custom `@action`s mirroring `archiveGlobalItem`, `restoreGlobalItem`, and `openMoveItemModal`'s split-quantity-and-move flow exactly.

```python
# views.py
from rest_framework import viewsets, status
from rest_framework.decorators import action
from rest_framework.response import Response
from django.utils import timezone
from .models import Location, Role, User, StorageUnit, Item, ItemLogEntry
from .serializers import (
    LocationSerializer, RoleSerializer, UserSerializer,
    StorageUnitSerializer, ItemSerializer, ItemLogEntrySerializer,
)
from .permissions import HasModulePermission


class LocationViewSet(viewsets.ModelViewSet):
    queryset = Location.objects.all()
    serializer_class = LocationSerializer
    permission_classes = [HasModulePermission]
    module = "rooms"

    def destroy(self, request, *args, **kwargs):
        location = self.get_object()
        if location.storage_units.exists():
            return Response(
                {"detail": "Move or delete its storage units first."},
                status=status.HTTP_409_CONFLICT,
            )
        return super().destroy(request, *args, **kwargs)


class RoleViewSet(viewsets.ModelViewSet):
    queryset = Role.objects.all()
    serializer_class = RoleSerializer
    permission_classes = [HasModulePermission]
    module = "roles"


class UserViewSet(viewsets.ModelViewSet):
    queryset = User.objects.all()
    serializer_class = UserSerializer
    permission_classes = [HasModulePermission]
    module = "users"


class StorageUnitViewSet(viewsets.ModelViewSet):
    queryset = StorageUnit.objects.all()
    serializer_class = StorageUnitSerializer
    permission_classes = [HasModulePermission]
    module = "storage"

    def get_queryset(self):
        qs = super().get_queryset()
        p = self.request.query_params
        if p.get("search"):
            qs = qs.filter(name__icontains=p["search"])
        if p.get("room"):
            qs = qs.filter(store_room_id=p["room"])
        if p.get("type"):
            qs = qs.filter(type=p["type"])
        return qs


class ItemViewSet(viewsets.ModelViewSet):
    queryset = Item.objects.all()
    serializer_class = ItemSerializer
    permission_classes = [HasModulePermission]
    module = "items"

    def get_queryset(self):
        qs = super().get_queryset()
        p = self.request.query_params
        if p.get("status"):
```

```
            qs = qs.filter(status=p["status"])
        if p.get("storageId"):
            qs = qs.filter(storage_id=p["storageId"])
        return qs

    @action(detail=True, methods=["post"])
    def archive(self, request, pk=None):
        item = self.get_object()
        item.status = "removed"
        item.save()
        ItemLogEntry.objects.create(
            id=f"LG-{int(timezone.now().timestamp())}", type="removed",
            item_name=item.name, quantity=item.quantity,
            rack=item.storage, rack_name=item.storage.name if item.storage else "",
            user=request.user.username,
        )
        return Response(self.get_serializer(item).data)

    @action(detail=True, methods=["post"])
    def restore(self, request, pk=None):
        item = self.get_object()
        item.status = "in_store"
        item.save()
        return Response(self.get_serializer(item).data)

    @action(detail=True, methods=["post"])
    def move(self, request, pk=None):
        """Splits off `quantity` units into `storageId` with `status`,
        mirroring openMoveItemModal's split/move behaviour (Section 4)."""
        item = self.get_object()
        qty = int(request.data["quantity"])
        target_storage_id = request.data.get("storageId")
        new_status = request.data.get("status", item.status)
        if qty >= item.quantity:
            item.storage_id = target_storage_id
            item.status = new_status
            item.save()
        else:
            item.quantity -= qty
            item.save()
            Item.objects.create(
                id=f"IT-{int(timezone.now().timestamp())}", name=item.name,
                tags=item.tags, quantity=qty, remarks=item.remarks,
                storage_id=target_storage_id, status=new_status,
            )
        return Response(self.get_serializer(item).data)


class ItemLogViewSet(viewsets.ReadOnlyModelViewSet):
    queryset = ItemLogEntry.objects.all()
    serializer_class = ItemLogEntrySerializer
    permission_classes = [HasModulePermission]
    module = "items"

    def get_queryset(self):
        qs = super().get_queryset()
        p = self.request.query_params
        if p.get("type"):
            qs = qs.filter(type=p["type"])
        if p.get("itemName"):
            qs = qs.filter(item_name__icontains=p["itemName"])
        if p.get("storageId"):
            qs = qs.filter(rack_id=p["storageId"])
        if p.get("user"):
            qs = qs.filter(user_id=p["user"])
        return qs
```

6.5 Authentication & URL Routing

Replaces the single hardcoded `admin` / `admin123` credential in `js/login.js` with a real token per `User` row via DRF's built-in `authtoken` app.

> `django.contrib.auth.authenticate()`, `TokenAuthentication`, and `request.user` all require the user model to be wired into Django's auth system — the exact `User` model in Section 6.1 is data-storage only and isn't yet. The smallest change that makes this code actually run: make `User` extend `django.contrib.auth.models.AbstractUser` instead of `models.Model`, add `AUTH_USER_MODEL = "storage_rack.User"` to `settings.py`, and drop the now-redundant `username` / `email` field declarations (`AbstractUser` already provides them, plus `password`, `is_staff`, `is_active`, etc. — fields beyond Section 2.4's exact shape, but required for login to work at all).

```
# auth_views.py
from rest_framework.views import APIView
from rest_framework.response import Response
from rest_framework.authtoken.models import Token
from django.contrib.auth import authenticate

class LoginView(APIView):
    permission_classes = []

    def post(self, request):
        user = authenticate(
            username=request.data.get("username"),
            password=request.data.get("password"),
        )
        if not user:
            return Response({"detail": "Invalid credentials."}, status=401)
        token, _ = Token.objects.get_or_create(user=user)
        return Response({
            "token": token.key,
            "user": {"id": user.id, "name": user.name, "roleId": user.role_id},
        })


class LogoutView(APIView):
    def post(self, request):
        request.user.auth_token.delete()
        return Response(status=204)
```

```
# urls.py
from django.urls import path, include
from rest_framework.routers import DefaultRouter
from .views import (
    LocationViewSet, RoleViewSet, UserViewSet,
    StorageUnitViewSet, ItemViewSet, ItemLogViewSet,
)
from .auth_views import LoginView, LogoutView

router = DefaultRouter()
router.register("locations", LocationViewSet)
router.register("roles", RoleViewSet)
router.register("users", UserViewSet)
router.register("storages", StorageUnitViewSet)
router.register("items", ItemViewSet)
router.register("log", ItemLogViewSet)

urlpatterns = [
    path("api/", include(router.urls)),
    path("api/auth/login/", LoginView.as_view()),
    path("api/auth/logout/", LogoutView.as_view()),
]
```

This router wiring produces exactly the endpoint set below — one resource per model, mirroring the existing internal "API" in Section 5 so each page's logic maps onto a matching endpoint instead of being redesigned.

RESOURCE	ENDPOINTS	REPLACES (CLIENT-SIDE)
Storage Units	`GET/POST /api/storages/` `GET/PUT/PATCH/DELETE /api/storages/{id}/`	`loadRacks` / `saveRacks`; `PATCH` backs the per-field click-to-edit on Storage Detail
Items	`GET/POST /api/items/` `GET/PUT/PATCH/DELETE /api/items/{id}/` `POST /api/items/{id}/archive/` · `/restore/` · `/move/`	`loadGlobalItems` / `saveGlobalItems`, `archiveGlobalItem`, `restoreGlobalItem`, `openMoveItemModal`'s split/move flow
Locations	`GET/POST /api/locations/` `PUT/DELETE /api/locations/{id}/`	`loadRooms` / `saveRooms` (delete returns 409 if any storage still references it, same guard `storerooms.js` already does client-side)
Users	`GET/POST /api/users/` `PUT/DELETE /api/users/{id}/` `POST /api/auth/login/` · `/logout/`	`loadUsers` / `saveUsers`; replaces the hardcoded demo login entirely
Roles	`GET/POST /api/roles/` `PUT/DELETE /api/roles/{id}/`	`loadRoles` / `saveRoles`
IN/OUT Log	`GET /api/log/` (read-only; filters: `type`, `itemName`, `storageId`, `user`)	`loadLog`; entries are written server-side as a side effect of item mutations, never POSTed directly by the client

6.6 Project Setup

```
# settings.py — essentials only
INSTALLED_APPS = [
    ...
    "django.contrib.auth",
    "django.contrib.contenttypes",
    "rest_framework",
    "rest_framework.authtoken",
    "corsheaders",
    "storage_rack",                          # this app
]

MIDDLEWARE = [
    "corsheaders.middleware.CorsMiddleware",
    ...
]

REST_FRAMEWORK = {
    "DEFAULT_AUTHENTICATION_CLASSES": [
        "rest_framework.authentication.TokenAuthentication",
    ],
    "DEFAULT_PERMISSION_CLASSES": [
        "rest_framework.permissions.IsAuthenticated",
    ],
}

CORS_ALLOWED_ORIGINS = ["http://localhost:8765"]  # wherever the existing frontend is served from
```

```
# requirements.txt
Django>=5.0
djangorestframework>=3.15
django-cors-headers>=4.3
```

No `Pillow`, `MEDIA_URL`, or `MEDIA_ROOT` needed to get started — `images` is a plain `JSONField` on `StorageUnit` / `Item` (Section 6.1), so base64 data-URIs round-trip exactly as the frontend already sends them, no file storage required. Add those later only if you choose to normalize into real file uploads (optional — see Section 6.7).

Standard bring-up from there: `django-admin startproject backend`, `python manage.py startapp storage_rack`, drop the files above into that app, then `python manage.py makemigrations storage_rack && python manage.py migrate && python manage.py`

`createsuperuser && python manage.py runserver`.

6.7 Migration Path

Keep the frontend changes small by preserving function names and JSON shapes end to end:

- Request/response bodies match the field names in Section 2 exactly — no reshaping needed in the page scripts that already consume these objects.
- Each `load*()` in `common.js` becomes a `fetch()` to its `GET` list endpoint; each `save*(data)` — which today rewrites the entire collection to `localStorage` on every single mutation — is replaced by the one targeted `POST` / `PUT` / `PATCH` / `DELETE` call that the mutation actually needs.
- `window.onRackDataChanged` / `window.onItemDataChanged` (Section 5) keep their exact role — just fired from a fetch `.then()` instead of right after a synchronous `localStorage.setItem`.
- `images` needs no migration at all day one — it's a `JSONField(list)` on the model (Section 6.1), so the frontend's existing base64 array round-trips through the API unchanged. Normalizing to real file uploads (a separate `StorageImage` / `ItemImage` model, `multipart/form-data`, returned URLs instead of inline data-URIs) is a worthwhile optimization once image volume grows, but it's an opt-in improvement, not a requirement to launch.